

Practical Assignment-2

Class, Object & Variables

1. **Create a class `Student`** with instance variables (name, rollNo, marks) and a method to display student details.
2. **Write a program** to create multiple objects of a class and access their members using object reference.

this and static Keyword

3. **Demonstrate the use of `this` keyword** to resolve variable shadowing in a class with a method.
4. **Create a class with static members** and methods. Show how static variables share common memory among objects.

Method Overloading

5. **Write a program with a `Calculator` class** where the `add()` method is overloaded for int, float, and double parameters.

final Keyword

6. **Create a class `ConstantValues`** with final variables and methods. Try to override a final method and observe the error.
7. **Use `final` with classes and variables** and explain its restrictions with comments.

Constructors

8. **Implement default and parameterized constructors** in a class `Book`. Initialize and display book details.
9. **Write a program to overload constructors** in a class `Rectangle` to calculate area based on input.
10. **Pass an object as a constructor parameter** to copy values from one object to another.

Inheritance (Single, Multilevel, Hierarchical)

11. **Write a program** for single inheritance using classes `Person` → `Employee`.
12. **Demonstrate multilevel inheritance** using `Animal` → `Mammal` → `Dog`.
13. **Create a program showing hierarchical inheritance** with a base class `Shape` and derived classes `Circle`, `Rectangle`.

super Keyword

14. Use **super** to call a superclass constructor and access overridden methods/variables from the subclass.

Method Overriding

15. Write a class **vehicle** with a method `run()`. Override it in **Bike** and **Car** classes and call them using object references.

Extending Class & Dynamic Method Dispatch

16. Create a base class **Appliance** and derived class **washingMachine**. Use dynamic method dispatch to invoke overridden methods.
17. Demonstrate **polymorphism** by storing subclass objects in superclass references and calling overridden methods.

Object Class Methods

18. Override `toString()` and `equals()` method in a class **Product** to compare and print meaningful object info.

Abstract Class

19. Define an abstract class **shape** with an abstract method `calculateArea()`. Implement it in **Circle** and **Square** classes.

Interface

20. Create an interface **Printable** with a method `print()`. Implement this in **Document** and **Image** classes.
21. Create multiple interfaces (**Movable**, **Stoppable**) and implement them in a class **Car**.